Checking Existence with Any() + All()

Boolean checks for existence and validation

1. Core idea

`Any()` and `All()` answer questions that produce a Boolean result (`true` or `false`) without retrieving a full list of entities.

Method	Meaning	Typical use
Any()	Returns true if at least one row matches. With a predicate, it checks whether any row satisfies that condition.	Does an email/student/enrollment exist?
All()	Returns true only if every row satisfies the condition.	Are all courses valid or all enrollments completed?

Mental anchor: Any() asks “is there at least one?”; All() asks “do all satisfy this rule?”

2. Short examples

ExistenceChecks.cs

```cs
bool hasActiveStudents = context.Students
    .Any(s => s.Status == "Active");

bool allCoursesValid = context.Courses
    .All(c => c.Price > 0);
```

3. How EF Core translates them

Any() is commonly translated using SQL `EXISTS`. The database only needs to establish whether a matching row exists.

Any_SQL.sql

```sql
SELECT CASE WHEN EXISTS (
    SELECT 1
    FROM [Students] AS [s]
    WHERE [s].[Status] = N'Active'
) THEN CAST(1 AS bit) ELSE CAST(0 AS bit) END
```

All() is commonly translated using `NOT EXISTS` over rows that violate the condition. If no violating row exists, the result is true.

Important: `All()` returns **true for an empty set** (vacuous truth). If at least one row must exist too, combine an existence check with the validation.

4. Common mistake: Count() > 0

Any_vs_Count.cs

```cs
// Less suitable for an existence check
bool exists = context.Students
    .Where(s => s.Status == "Active")
    .Count() > 0;

// Prefer Any()
bool exists = context.Students
    .Any(s => s.Status == "Active");
```

Use `Any()` when you only need yes/no. `Count() > 0` asks for a count that you do not actually need; `Any()` communicates the intent directly and is typically more efficient.

5. Practical examples

• **Any():** check whether an email is already used, whether a student exists, or whether a course has enrollments.
• **All():** verify that every invoice is paid, every course has a positive price, or every enrollment is completed.

Next lesson: aggregation methods such as Count(), Sum(), and Average().